

Project Kisan: Backend Architecture Overview

Core Idea

Project Kisan uses **Vertex AI Agent Builder** as the central brain that dynamically orchestrates multiple AI services and tools. The backend, built using **FastAPI and deployed on Cloud Run**, serves as the gateway for user inputs and facilitates communication with various AI tools and the Agent Builder.

System Workflow Summary

Frontend (Web-Based Interface)

- Provides interfaces for users (farmers) to input queries via **voice, image, or text**.
- Sends input to the backend using HTTP or **WebSockets** for real-time communication.

Backend (FastAPI on Cloud Run)

1. Gateway Layer

2. Receives all user inputs.
3. Dispatches inputs to appropriate processing modules.

4. Voice Handler

5. **ASR (Speech-to-Text)**: Raw audio is sent to **Vertex AI Speech-to-Text** for transcription.

6. **Agent Interaction**: Transcribed text is passed to Agent Builder.

7. **TTS (Text-to-Speech)**: Final response is sent to **Vertex AI Text-to-Speech** and streamed back as audio.

8. Agent Orchestrator

9. Accepts either raw text (direct input or from ASR).
 10. Sends query to **Vertex AI Agent Builder** via SDK.
 11. Awaits structured response from the agent.
-

Agent Builder (Using Vertex AI Agent SDK)

- **Intent Recognition**: Uses **Gemini Pro** model to identify user's intent.
- **Tool Selection**: Based on intent and metadata, selects appropriate backend service/tool.

Backend Tools (Cloud Run or Cloud Functions)

Custom RAG System (Knowledge Retrieval)

Function: `query_agricultural_knowledge(query_text, corpus_type)`

Purpose: Retrieves information using a manually built RAG pipeline that queries preprocessed files stored in

Corpora: `disease_causes`, `remedies`, `government_policies`, `general_queries`

Deployment: Cloud Run/Cloud Function; exposed to Agent Builder via REST interface.

Image Analysis Tool

Function: `diagnose_plant_image(image_base64)`

Steps:

- Sends image to Google Cloud Vision API or a fallback trained ML model for disease classification.
- Extracts potential disease(s).
- Triggers `query_agricultural_knowledge` with `corpus_type="remedies"` for recommended solutions.
- Gemini Pro Text composes the final diagnosis + remedy response.

Deployment: Cloud Run; exposed to Agent Builder.

Voice Integration Stack

- **STT (Speech-to-Text)**: Vertex AI Speech.
 - **TTS (Text-to-Speech)**: Vertex AI Speech.
 - Real-time voice streaming via **WebSockets** (if supported by model).
-

Google Cloud CLI & Agent SDK Setup (Required)

1. **Install gcloud CLI**: <https://cloud.google.com/sdk/docs/install>
2. **Authenticate**:

```
gcloud auth login
gcloud config set project [PROJECT_ID]
```

3. **Enable Required APIs**:

```
gcloud services enable aiplatform.googleapis.com \
    speech.googleapis.com \
    texttospeech.googleapis.com \
    run.googleapis.com \
    cloudfunctions.googleapis.com
```

4. **Install Agent SDK**:

```
pip install google-cloud-aiplatform
pip install vertexai-agent-sdk
```

5. **Initialize SDK**:

```
from vertexai.preview.agentbuilder import Agent
agent = Agent(project="your-project-id", location="us-central1")
```

Summary: Centralized Intelligent Orchestration

- **Frontend** collects multimodal inputs.
- **FastAPI backend** dispatches them to tools.
- **Vertex AI Agent Builder** acts as the orchestrator.
- **Gemini Pro + RAG Engine + APIs** deliver smart, friendly, contextual answers.
- **Real-time interaction** supported via STT/TTS and WebSockets.

Project Kisan creates a holistic digital assistant for farmers by intelligently combining AI tools, Vertex services, and Google Cloud deployment strategies.